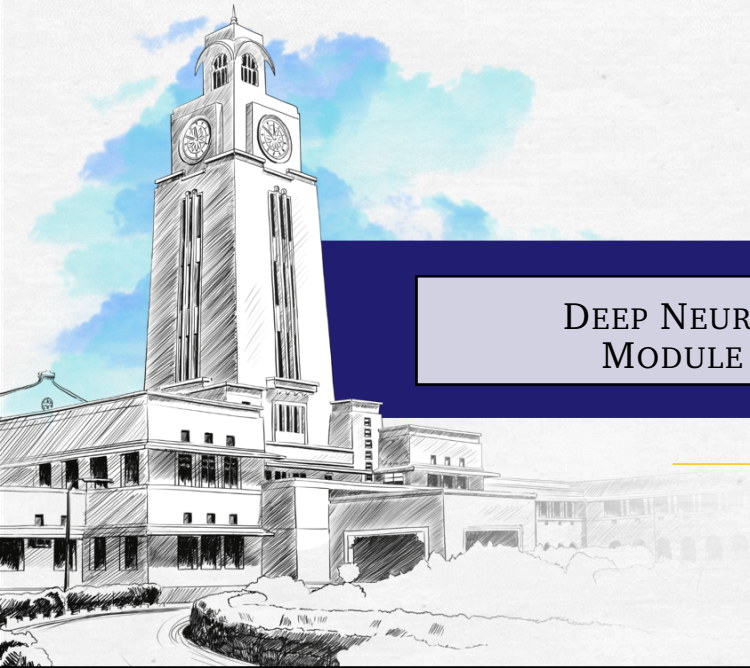




BITS Pilani
Pilani | Dubai | Goa | Hyderabad

DEEP NEURAL NETWORK MODULE # 6: CNN

DNN Team
BITS Pilani WILP



With grateful acknowledgement to the authors who have
generously made their course materials freely available online.
Deck prepared by Seetha Parameswaran.

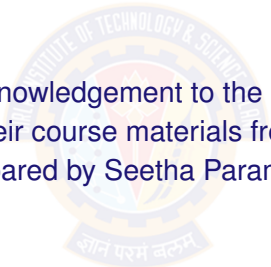


TABLE OF CONTENTS

- 1 COMPUTER VISION TASKS
- 2 IMAGE BASICS AND IMAGE CLASSIFICATION CHALLENGES
- 3 INTRODUCTION TO CONVOLUTIONAL NETWORKS
- 4 CNN COMPONENTS
- 5 CNN ARCHITECTURE DESIGN
- 6 POPULAR CNN ARCHITECTURES
- 7 TRANSFER LEARNING
- 8 CNN APPLICATIONS
- 9 IMPLEMENTATION CONSIDERATIONS
- 10 SUMMARY



IMAGE CLASSIFICATION



64x64 x3

→ Cat? (0/1)



→ Cat? (0/1)



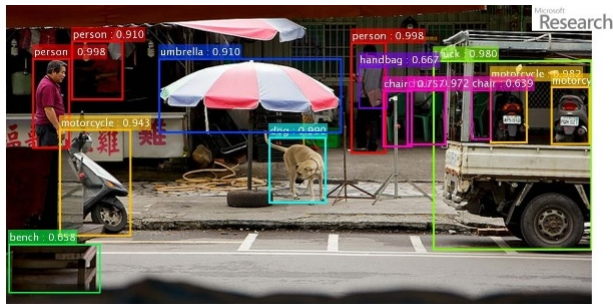
Classify into 3 classes

FACE DETECTION



Fei-Fei Li & Justin Johnson & Serena Yeung

OBJECT RECOGNITION



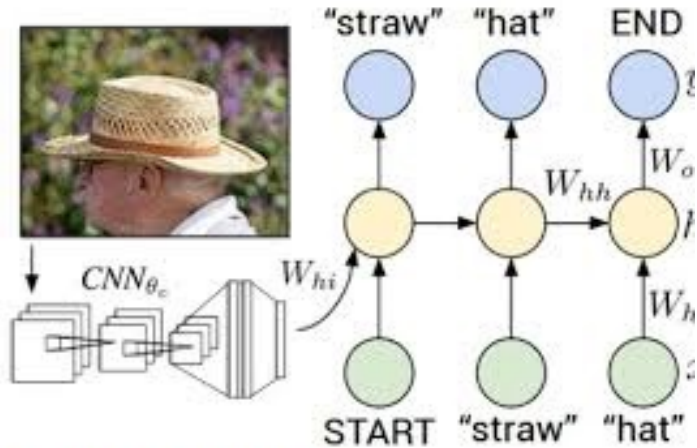
*the original image is from the COCO dataset

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition", arXiv 2015.

Shaoqing Ren, Kaiming He, Ross Girshick, & Jian Sun. "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015.

Derin-Renme-Nedir-Akademik-Bilim-2016

IMAGE CAPTIONING



Andrej Karpathy, Fei-Fei Li

TABLE OF CONTENTS

- 1 COMPUTER VISION TASKS
- 2 IMAGE BASICS AND IMAGE CLASSIFICATION CHALLENGES
- 3 INTRODUCTION TO CONVOLUTIONAL NETWORKS
- 4 CNN COMPONENTS
- 5 CNN ARCHITECTURE DESIGN
- 6 POPULAR CNN ARCHITECTURES
- 7 TRANSFER LEARNING
- 8 CNN APPLICATIONS
- 9 IMPLEMENTATION CONSIDERATIONS
- 10 SUMMARY



DIGITAL GRAYSCALE IMAGE



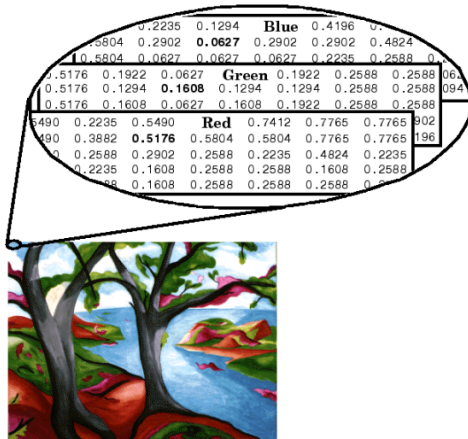
Pixel intensities = $I(x, y)$

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	14	1	0	6	6	0	0	0

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	14	1	0	6	6	0	0	0

<https://mozanunal.com/2019/11/img2sh/>

DIGITAL COLOR IMAGE



https://www.mathworks.com/help/matlab/creating_plots/image-types.html

IMAGE CLASSIFICATION - CHALLENGES

Image Classification: A core task in Computer Vision



This image by Nikita is
licensed under [CC-BY 2.0](#)

(assume given set of discrete labels)
{dog, cat, truck, plane, ...}

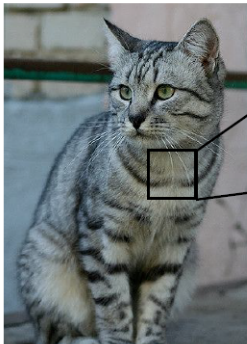


cat

Fei-Fei Li & Justin Johnson & Serena Yeung

IMAGE CLASSIFICATION - CHALLENGES

The Problem: Semantic Gap



This image by Nikita is
licensed under [CC-BY 2.0](#)

```
[185 112 100 111 104 95 106 99 96 103 112 119 104 97 93 87]  
[ 91 98 102 106 104 79 90 183 99 105 123 136 118 105 94 85]  
[ 76 85 90 105 128 105 87 96 95 99 115 112 106 103 99 85]  
[ 99 81 81 93 120 131 127 180 95 98 102 99 96 93 181 94]  
[106 91 61 64 69 91 88 85 181 107 109 98 75 84 96 95]  
[114 188 85 55 55 69 64 54 64 87 112 129 98 74 84 91]  
[133 137 147 103 65 81 88 65 52 54 74 84 182 93 85 82]  
[128 137 144 140 109 95 86 70 62 65 63 63 68 73 86 181]  
[129 133 148 137 119 121 117 94 65 79 80 65 54 64 72 98]  
[127 125 131 147 133 127 126 131 111 96 89 75 61 64 72 84]  
[115 114 109 123 150 148 131 118 113 109 100 92 74 65 72 78]  
[ 89 93 90 97 108 147 131 118 113 114 113 109 106 95 77 89]  
[ 63 77 86 81 77 79 182 123 117 115 117 125 125 138 115 87]  
[ 62 65 82 89 78 71 88 181 124 126 119 101 107 114 131 119]  
[ 63 65 75 88 89 71 62 81 120 138 135 105 81 90 110 118]  
[ 87 65 71 87 106 95 60 45 76 130 126 107 92 94 185 112]  
[119 97 82 86 117 123 116 56 41 51 95 93 89 95 182 107]  
[164 146 112 80 82 128 124 184 76 48 45 66 88 101 182 189]  
[157 170 157 120 93 86 114 132 112 97 69 55 78 82 99 94]  
[138 128 134 161 139 188 109 118 121 134 114 87 65 53 69 86]  
[128 112 96 117 150 144 128 115 184 107 102 93 87 81 72 79]  
[123 187 96 86 83 112 153 149 122 109 104 75 88 107 112 99]  
[122 121 102 80 82 86 94 117 145 148 153 102 58 70 92 187]  
[122 164 148 103 71 56 78 83 93 103 119 139 102 61 69 84]
```

What the computer sees

An image is just a big grid of
numbers between [0, 255]:

e.g. 800 x 600 x 3
(3 channels RGB)

Fei-Fei Li & Justin Johnson & Serena Yeung

IMAGE CLASSIFICATION - CHALLENGES

Pixelated Images



Larger Images



- A 50x50 Image requires 2500 input dimensions.
- A HD image 1920x1080 is about 2 million inputs. (2 Mega Pixels)

IMAGE INVARIANCE - CHALLENGES

Object can appear anywhere in the image.

Image A

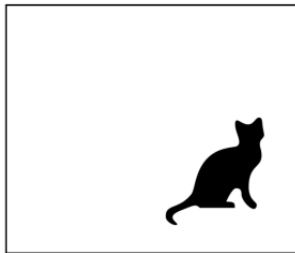
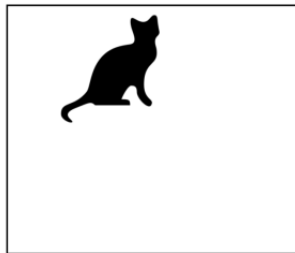


Image B

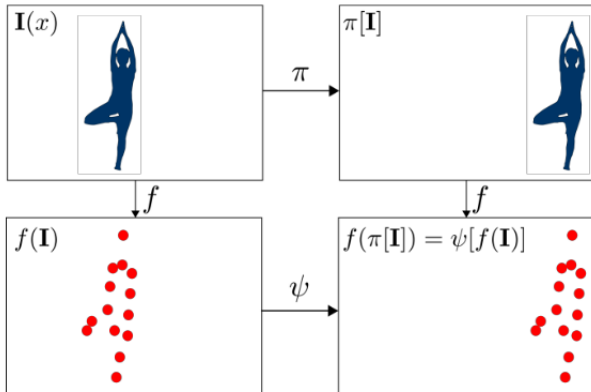


Model predicts “cat” for both images!

Source: <https://penkovsky.com/neural-networks/day5/>

IMAGE EQUIVARIANCE - CHALLENGES

Changes to the input predictably (equivalently) should not affect the output.



Source: <https://towardsdatascience.com/translational-invariance-vs-translational-equivariance-f9fbc8fca63a>

IMAGE CLASSIFICATION - SOLUTION

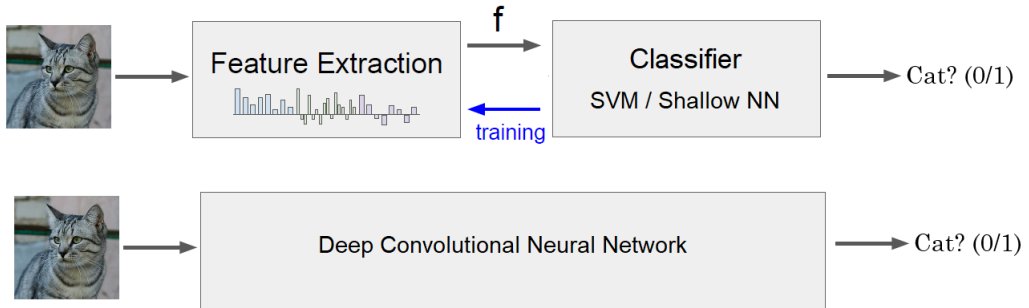


TABLE OF CONTENTS

- 1 COMPUTER VISION TASKS
- 2 IMAGE BASICS AND IMAGE CLASSIFICATION CHALLENGES
- 3 INTRODUCTION TO CONVOLUTIONAL NETWORKS**
- 4 CNN COMPONENTS
- 5 CNN ARCHITECTURE DESIGN
- 6 POPULAR CNN ARCHITECTURES
- 7 TRANSFER LEARNING
- 8 CNN APPLICATIONS
- 9 IMPLEMENTATION CONSIDERATIONS
- 10 SUMMARY



WHAT IS A CONVOLUTIONAL NEURAL NETWORK?

DEFINITION

A Convolutional Neural Network (CNN) is a specialized type of deep neural network designed to process data with a grid-like topology, such as images, where spatial relationships between data points are important.

CORE INSIGHT

CNNs mimic the visual cortex by detecting simple patterns (edges, corners) in early layers and combining them into complex objects (faces, cars) in deeper layers.

WHY CONVOLUTIONAL NETWORKS?

WHY CNNs FOR IMAGES?

- **Translation Invariance:** Object recognition regardless of position
- **Local Patterns:** Detect edges, textures, and shapes locally
- **Hierarchical Features:** Low-level → Mid-level → High-level features
- **Parameter Efficiency:** Fewer parameters than fully connected networks
- **Computational Efficiency:** Parallel processing of convolutions

WHY CNN FOR IMAGES - LOCALITY

- Some patterns are much smaller than the whole image.



Courtesy: Akka Zemhari

WHY CNN FOR IMAGES - LOCALITY

- Some patterns are much smaller than the whole image.

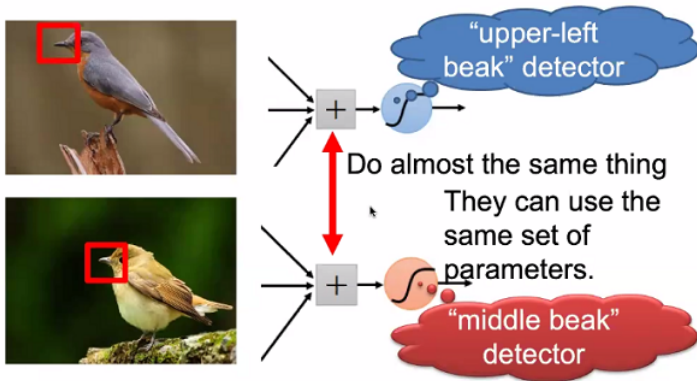
A neuron does not have to see the whole image to discover the pattern.



Courtesy: Akka Zemhari

WHY CNN FOR IMAGES - TRANSLATION INVARIANCE

- Same pattern may appear in different regions.



Courtesy: Akka Zemhari

WHY CNN FOR IMAGES - SCALE INVARIANCE

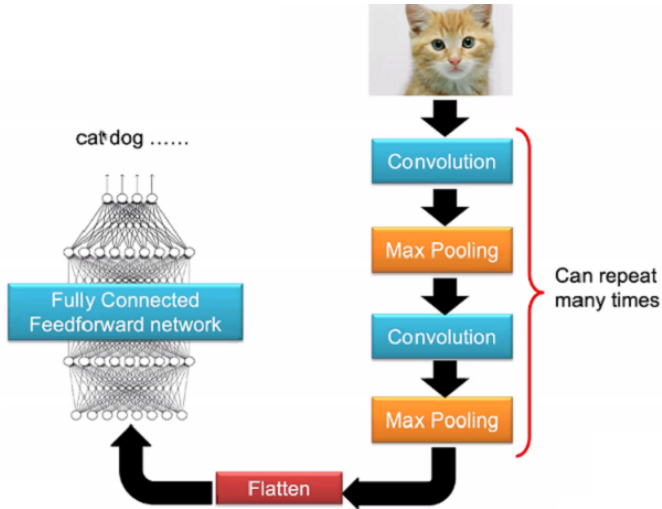
- Sub-sampling the pixels will not change the object.



- We can sub-sample the image to make it smaller.
- Less parameters for the network to process the image.

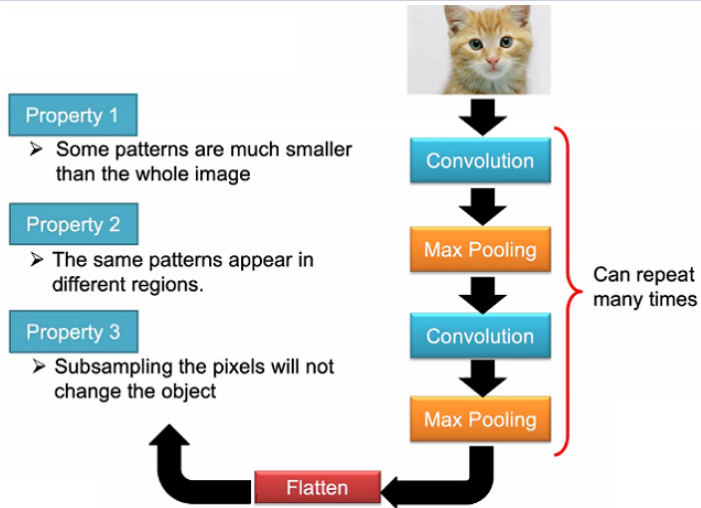
Courtesy: Akka Zemhari

THE WHOLE CNN



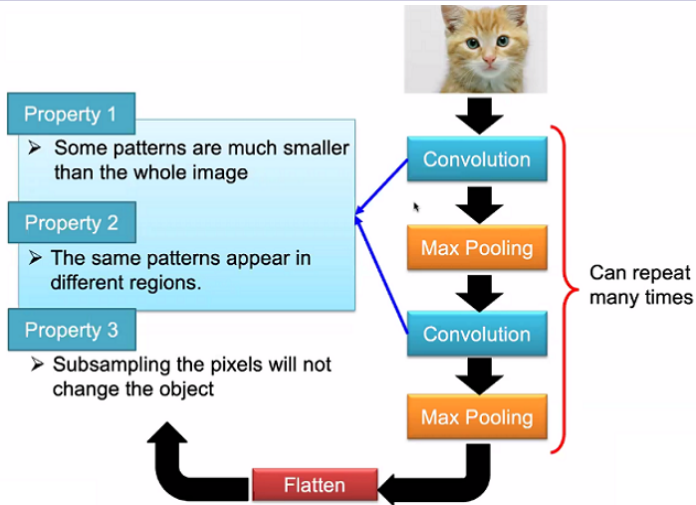
Courtesy: Akka Zemmari

THE WHOLE CNN



Courtesy: Akka Zemhari

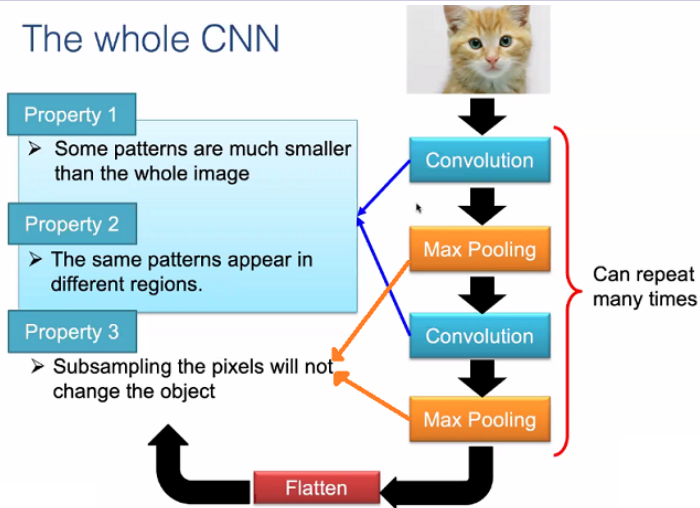
THE WHOLE CNN



Courtesy: Akka Zemmari

THE WHOLE CNN

The whole CNN



Courtesy: Akka Zemmari

KEY PRINCIPLES OF CNNs

THREE KEY PRINCIPLES

① Local Connectivity (Sparse Interactions)

- ▶ Each neuron connects only to a local region of the input
- ▶ Receptive field concept

② Parameter Sharing

- ▶ Same filter/kernel used across entire input
- ▶ Dramatically reduces parameters

③ Equivariant Representations

- ▶ If input shifts, output shifts correspondingly
- ▶ Built-in translation equivariance

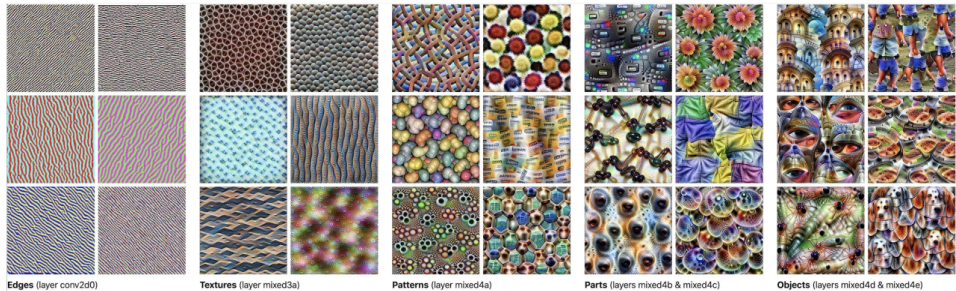
CNN KEY CHARACTERISTICS

- **Convolutional Layers:** Apply learnable filters to detect local features
- **Pooling Layers:** Reduce spatial dimensions and provide translation invariance
- **Hierarchical Learning:** Build complex features from simple ones
- **Parameter Sharing:** Same filter applied across entire input
- **Sparse Connectivity:** Each neuron connects to local region only

How CNN Achieves Image Classification

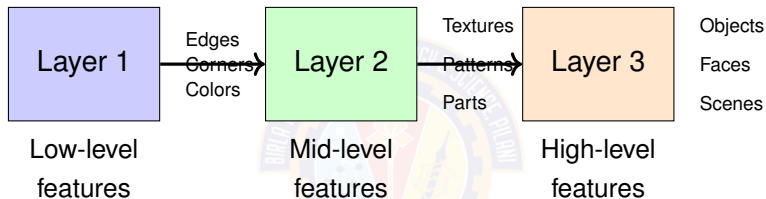
HIERARCHICAL FEATURE LEARNING PROCESS

CNNs achieve image classification through a hierarchical feature extraction process, where each layer builds increasingly complex representations from simpler ones.



HIERARCHICAL FEATURE LEARNING

Deep networks learn increasingly complex features:



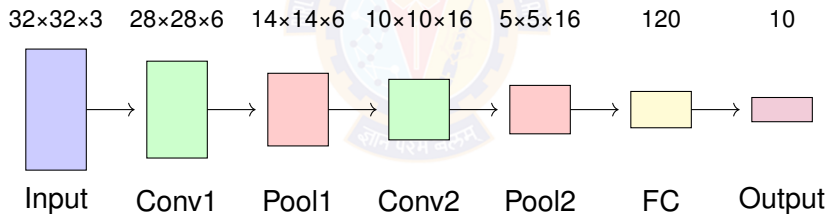
Example: Face Detection

- Layer 1: Detects edges at various orientations
- Layer 2: Combines edges into facial features (eyes, nose, mouth)
- Layer 3: Combines features into complete face

CNN ARCHITECTURE

ARCHITECTURE PATTERN

INPUT $\rightarrow$ [CONV $\rightarrow$ ACTIVATION $\rightarrow$ POOL] $\times$ N $\rightarrow$ FC $\rightarrow$ OUTPUT



CNN - LAYER-BY-LAYER BREAKDOWN

Early Layers (1-2)

- Purpose: Detect low-level features
- Features: Edges, corners, lines
- Filters: Small (3×3 , 5×5)
- Receptive Field: Local, small regions

Middle Layers (3-4)

- Purpose: Combine low-level features
- Features: Shapes, textures, patterns
- Complexity: Medium-level abstractions
- Receptive Field: Larger regions

CNN - LAYER-BY-LAYER BREAKDOWN

Deep Layers (5+)

- Purpose: Detect object parts
- Features: Wheels, faces, eyes, ears
- Complexity: High-level semantics
- Receptive Field: textcolorBitsRed image regions

Fully Connected Layers

- Purpose: Final classification decision
- Input: Feature vector from conv layers
- Output: Class probability scores
- Function: Combine all learned features

THE CLASSIFICATION PROCESS

- ① **Feature Extraction:** Convolutional layers progressively extract features of increasing complexity
- ② **Spatial Reduction:** Pooling layers reduce spatial dimensions while preserving important information
- ③ **Feature Combination:** Deeper layers combine simpler features to recognize complex patterns
- ④ **Decision Making:** Fully connected layers use extracted features to make final classification decision
- ⑤ **Output:** Softmax layer produces probability distribution over classes

KEY INSIGHT

Each layer acts as a **feature detector** that becomes increasingly sophisticated. Early layers detect simple patterns, while later layers detect complex objects by combining simpler patterns.

TABLE OF CONTENTS

- 1 COMPUTER VISION TASKS
- 2 IMAGE BASICS AND IMAGE CLASSIFICATION CHALLENGES
- 3 INTRODUCTION TO CONVOLUTIONAL NETWORKS
- 4 CNN COMPONENTS**
- 5 CNN ARCHITECTURE DESIGN
- 6 POPULAR CNN ARCHITECTURES
- 7 TRANSFER LEARNING
- 8 CNN APPLICATIONS
- 9 IMPLEMENTATION CONSIDERATIONS
- 10 SUMMARY



2D CONVOLUTION OPERATION

CROSS-CORRELATION (USED IN DEEP LEARNING)

$$(I \star K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

In practice, deep learning frameworks implement cross-correlation but call it "convolution"

- **Input:** Feature map/image I
- **Kernel/Filter:** Learnable parameters K
- **Output:** Feature map showing detected patterns

2D CONVOLUTION OPERATION

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6×6

These values inside the matrix are to be learned by the network. We call these as parameters or weights.

1	-1	-1
-1	1	-1
-1	-1	1

Kernel / Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Kernel / Filter 2

Property 1: Each filter detects a small pattern (3×3)

CONVOLUTION OPERATION

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6×6

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Start at top left corner.
Perform element wise
product and add all the
values.

Output

3

CONVOLUTION OPERATION

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6×6

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Slide by one pixel towards right.
Perform element wise product and add all the values.

Output

3

-1

CONVOLUTION OPERATION

stride $s = 1$

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6×6

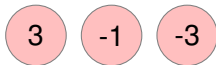
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Slide by one pixel
towards right.

Perform element wise
product and add all the
values.

Output



CONVOLUTION OPERATION

stride $s = 1$

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6×6

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Slide by one pixel
towards right.

Perform element wise
product and add all the
values.

Output

3

-1

-3

-1

CONVOLUTION OPERATION

stride $s = 1$

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

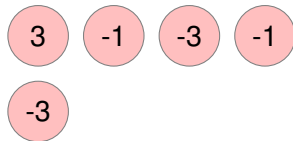
Input Image 6×6

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Jump to next row, left most pixel.
The filter has to completely superimpose the pixels of input.
NOT PARTIALLY.

Output



CONVOLUTION OPERATION

stride $s = 1$

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6×6

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

After sliding through the entire image.

3	-1	-3	-1
-3	-1	0	-3
-3	-2	0	1
3	-2	-2	-1

Output Feature Map

CONVOLUTION OPERATION

stride $s = 1$

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6×6

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

3	-1	-3	-1
-3	-1	0	-3
-3	-2	0	1
3	-2	-2	-1

Output Feature Map

Property 2 is satisfied.

CONVOLUTION OPERATION

stride $s = 1$

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

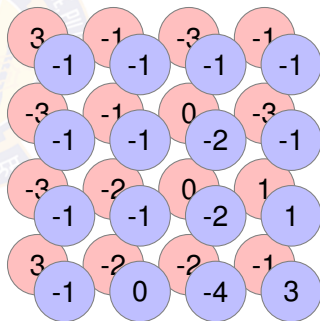
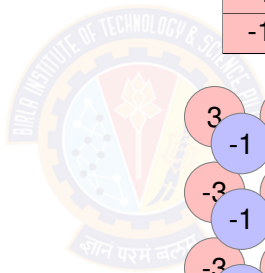
Input Image 6×6

Filter 1

1	-1	-1
-1	1	-1
-1	-1	1

Filter 2

-1	1	-1
-1	1	-1
-1	1	-1



Feature

Maps

WHAT IS A FEATURE MAP?

DEFINITION

A **feature map** (or activation map) is the output produced when a convolutional filter is applied to an input image or previous layer's output.

- **Key Concepts:**

- ▶ Each filter detects a specific pattern (edges, textures, shapes)
- ▶ Output of convolution = feature map showing **where** pattern was detected
- ▶ High values in feature map = strong pattern presence
- ▶ Low values = weak or absent pattern

- **Mathematical Form:**

$$\text{Feature Map}_k(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K_k(m, n) + b_k$$

where K_k is the k -th filter and b_k is its bias.

FEATURE MAPS: VISUAL EXAMPLE

Input Image

1	0	0	0	0	0
0	1	0	1	1	1
0	0	1	0	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Different Filters → Different Feature Maps

Filter 1: Vertical Edge Detector

-1	0	1
-1	0	1
-1	0	1

⇒ Vertical edges highlighted

Filter 2: Horizontal Edge Detector

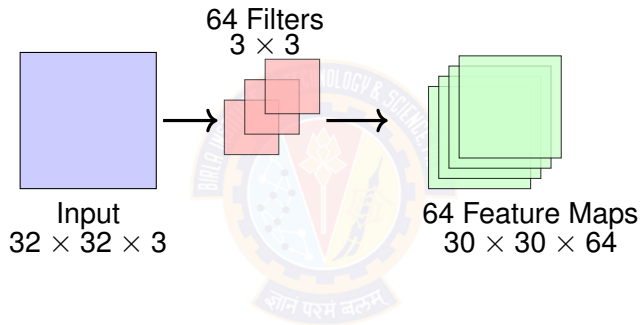
-1	-1	-1
0	0	0
1	1	1

⇒ Horizontal edges highlighted

Key Insight: Different filters create different feature maps, each capturing different aspects of the input.

MULTIPLE FEATURE MAPS IN CNNs

Single Convolutional Layer with Multiple Filters:



Important:

- Number of filters = Number of output feature maps
- Each feature map = One filter's response across entire input
- Stack of feature maps = 3D volume (height $\times$ width $\times$ channels)

RECEPTIVE FIELD: DEFINITION

DEFINITION

The **receptive field** of a neuron is the region in the input image that affects that neuron's activation value.

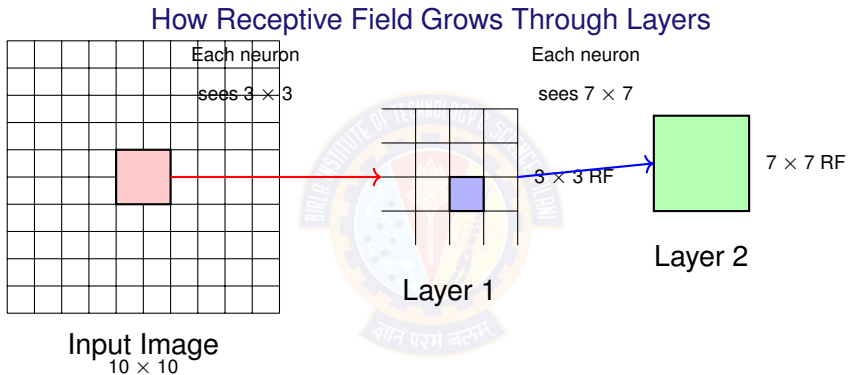
- Key Concepts:

- ▶ Receptive field grows with network depth
- ▶ Early layers: small receptive fields (local patterns)
- ▶ Deep layers: large receptive fields (global context)
- ▶ Determined by kernel size, stride, and pooling

- Importance:

- ▶ Small receptive field: Cannot capture large objects
- ▶ Large receptive field: Better context understanding
- ▶ Trade-off: Computational cost vs. context

RECEPTIVE FIELD: VISUAL EXAMPLE



Pattern: Receptive field size approximately doubles with each layer (depends on architecture).

RECEPTIVE FIELD: CALCULATION

Formula for receptive field in layer l :

$$RF_l = RF_{l-1} + (K_l - 1) \times \prod_{i=1}^{l-1} S_i$$

- RF_l = Receptive field at layer l
- K_l = Kernel size at layer l
- S_i = Stride at layer i

Example: Sequential 3×3 convolutions with stride 1

Layer 1: $RF_1 = 3$

Layer 2: $RF_2 = 3 + (3 - 1) \times 1 = 5$

Layer 3: $RF_3 = 5 + (3 - 1) \times 1 = 7$

Layer 4: $RF_4 = 7 + (3 - 1) \times 1 = 9$

Key Insight: Stacking small kernels (3×3) is more efficient than using large kernels (9×9) while achieving the same receptive field!

LEARNING A KERNEL: OVERVIEW

KEY QUESTION

How does the network learn what values to put in the convolutional kernels/filters?

Answer: Through Gradient Descent!

- Kernel weights are **learnable parameters** (like weights in fully connected layers)
- Initialized randomly (or with specific strategies)
- Updated via backpropagation and gradient descent
- Network learns kernels that minimize the loss function

Process:

Random kernels $\xrightarrow{\text{Training}}$ Meaningful edge/texture/pattern detectors

KERNELS AS PARAMETERS

- Convolutional layer parameters:

For a filter of size $K \times K$ with C_{in} input channels:

$$\text{Parameters per filter} = K \times K \times C_{in} + 1$$

(The $+1$ is for the bias term)

- Example: 3×3 filter, 64 input channels

- ▶ Weights: $3 \times 3 \times 64 = 576$
- ▶ Bias: 1
- ▶ Total: 577 learnable parameters per filter

- Gradient Descent Update:

$$K_{new} = K_{old} - \eta \frac{\partial \mathcal{L}}{\partial K}$$

where $\mathcal{L}$ is the loss function and η is the learning rate.

WHAT DO LEARNED KERNELS LOOK LIKE?

Layer 1: Low-level feature detectors

Vertical Edge

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Horizontal Edge

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Diagonal Edge

$$\begin{bmatrix} 0 & -1 & -1 \\ 1 & 0 & -1 \\ 1 & 1 & 0 \end{bmatrix}$$

Characteristics of learned kernels:

- First layers: Edge detectors at various orientations and scales
- Middle layers: Texture and pattern detectors (blob detectors, corner detectors)
- Deep layers: Complex pattern detectors (harder to visualize)

Important: The network discovers these patterns automatically from data!

LEARNING KERNELS: BACKPROPAGATION THROUGH

- Forward Pass:

$$\text{Output}(i, j) = \sum_m \sum_n \text{Input}(i + m, j + n) \cdot K(m, n) + b$$

- Backward Pass (Gradient Calculation):

Gradient w.r.t. kernel weights:

$$\frac{\partial \mathcal{L}}{\partial K(m, n)} = \sum_i \sum_j \frac{\partial \mathcal{L}}{\partial \text{Output}(i, j)} \cdot \text{Input}(i + m, j + n)$$

Gradient w.r.t. bias:

$$\frac{\partial \mathcal{L}}{\partial b} = \sum_i \sum_j \frac{\partial \mathcal{L}}{\partial \text{Output}(i, j)}$$

Key Insight: Gradient computation is also a convolution-like operation!

PADDING

Why Padding?

- Without padding: Output size = Input size - Kernel size + 1
- Information at borders gets less attention
- Deep networks would shrink feature maps rapidly

Types of Padding

- Valid Padding: No padding (output smaller than input)
- Same Padding: Pad to keep output size = input size
- Full Padding: Maximum padding

SAME PADDING CALCULATION

For kernel size k and stride $s = 1$: $\text{Padding} = (k - 1)/2$

CONVOLUTION WITH PADDING

Input Image with padding 8×8

0	0	0	0	0	0	0	0
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	1	0	0	0	1	0	0
0	0	1	0	0	1	0	0
0	0	0	1	0	1	0	0
0	0	0	0	0	0	0	0

*

Filter1

1	-1	-1
-1	1	-1
-1	-1	1

STRIDE

Stride determines how much the kernel moves at each step.

- **Stride = 1**: Move kernel one pixel at a time (standard)
- **Stride = 2**: Move kernel two pixels at a time (downsampling)
- **Stride > 1**: Reduces output size, acts as downsampling

OUTPUT SIZE CALCULATION

For input size n , kernel size k , padding p , stride s :

$$\text{Output size} = \frac{n + 2p - k}{s} + 1$$

Result must be an integer for valid convolution. So take floor of the result.

CONVOLUTION OPERATION WITH STRIDE

stride $s = 3$

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6×6

1	-1	-1
-1	1	-1
-1	-1	1

Filter1

3	-1
3	-1

Feature Map 2×2

STRIDED CONVOLUTION

- Standard Stride ($s = 1$): Move filter one pixel at a time
- **Strided Convolution** ($s > 1$): Skip pixels to downsample
- Output size formula:

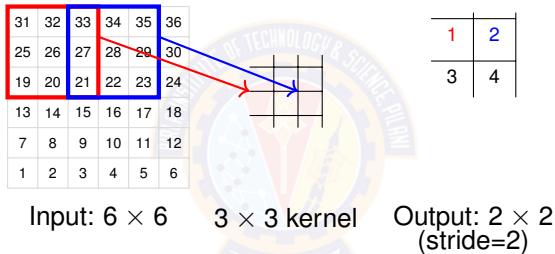
$$O = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

where n = input size, p = padding, k = kernel size, s = stride

- **Purpose of Strided Convolutions:**
 - ▶ Spatial downsampling: Reduce feature map size
 - ▶ Replace pooling: Learn downsampling instead of fixed operation
 - ▶ Computational efficiency: Fewer computations in subsequent layers
 - ▶ Increase receptive field: Larger context per neuron

STRIDED CONVOLUTION: EXAMPLE

Stride = 2 Example:



With stride = 2: Output size = $\frac{6-3}{2} + 1 = 2$

Trade-off: Lose spatial resolution but gain computational efficiency

DILATED (ATROUS) CONVOLUTION

DEFINITION

Dilated convolution (also called atrous convolution) inserts spaces (holes) between kernel elements to increase receptive field without increasing parameters.

Dilation rate r : Number of pixels between kernel elements

- $r = 1$: Standard convolution (no dilation)
- $r = 2$: One pixel gap between kernel elements
- $r = 3$: Two pixel gaps between elements

Effective receptive field:

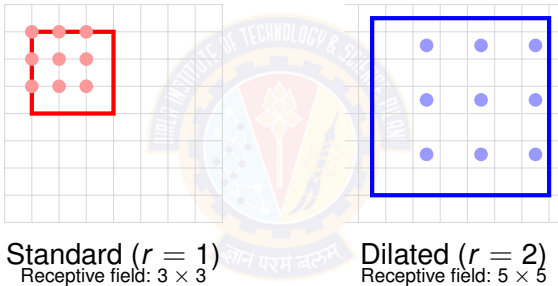
$$k_{\text{eff}} = k + (k - 1)(r - 1)$$

where k = kernel size, r = dilation rate

Example: 3×3 kernel with $r = 2 \rightarrow$ effective 5×5 receptive field

DILATED CONVOLUTION: VISUAL EXAMPLE

Standard vs. Dilated Convolution



Key Advantage: Same number of parameters (9), but larger receptive field!

DILATED CONVOLUTION: APPLICATIONS AND BENEFITS

- When to Use Dilated Convolutions:
 - ▶ Semantic segmentation: Need large receptive field while maintaining resolution
 - ▶ Dense prediction tasks: Object detection, depth estimation
 - ▶ Audio processing: WaveNet uses dilated convolutions for sequence modeling
- Benefits:
 - ① Larger receptive field without pooling or striding
 - ② Preserve spatial resolution (no downsampling)
 - ③ Same parameter count as standard convolution
 - ④ Multi-scale context by using different dilation rates
- Trade-offs:
 - ▶ May lose fine-grained local information
 - ▶ Gridding artifacts possible (mitigated with varying dilation rates)
 - ▶ Slightly more memory usage during forward pass

STRIDED VS. DILATED CONVOLUTION: COMPARIS

Property	Strided Conv	Dilated Conv
Output size	Reduced	Same as input
Receptive field	Increased	Increased
Parameters	Same	Same
Downsampling	Yes	No
Resolution loss	Yes	No
Typical use	Classification	Segmentation

Summary:

- **Strided convolution:** Reduce spatial dimensions, typically for classification
- **Dilated convolution:** Increase receptive field while maintaining resolution, for dense predictions
- Both are alternatives to pooling for different purposes

MULTIPLE CHANNELS: INTRODUCTION

Reality Check

Real images have **multiple channels**:

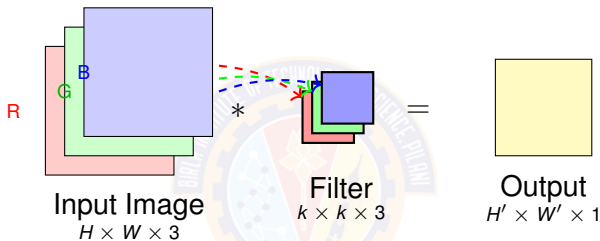
- RGB images: 3 channels (Red, Green, Blue)
- Grayscale: 1 channel
- Feature maps in CNNs: Typically 64, 128, 256, or 512 channels

Questions:

- 1 How does convolution work with multi-channel inputs?
- 2 How do we get multi-channel outputs?
- 3 How many parameters do we need?

MULTI-CHANNEL INPUT: SINGLE FILTER

Convolution with 3-channel input (e.g., RGB image):



Key Point: Filter has same depth as input!

$$\text{Output}(i, j) = \sum_{c=1}^3 \sum_m \sum_n I(i+m, j+n, c) \cdot K(m, n, c) + b$$

MULTI-CHANNEL CONVOLUTION: MATHEMATICAL

- For input $I \in \mathbb{R}^{H \times W \times C_{in}}$ and kernel $K \in \mathbb{R}^{k \times k \times C_{in}}$:
- Step 1: For each channel c , compute 2D convolution:

$$O_c(i, j) = \sum_m \sum_n I(i + m, j + n, c) \cdot K(m, n, c)$$

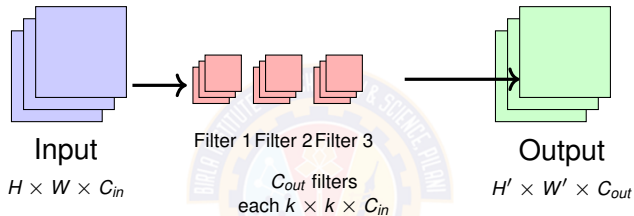
- Step 2: Sum across all input channels:

$$O(i, j) = \sum_{c=1}^{C_{in}} O_c(i, j) + b = \sum_{c=1}^{C_{in}} \sum_m \sum_n I(i + m, j + n, c) \cdot K(m, n, c) + b$$

- Result: One filter produces one output channel (feature map)
- Parameters per filter: $k \times k \times C_{in} + 1$ (bias)
- Example: 3×3 filter, 64 input channels = $3 \times 3 \times 64 + 1 = 577$ parameters

MULTI-CHANNEL INPUT AND OUTPUT

To get multiple output channels: Use multiple filters!



General Rule:

Input : $H \times W \times C_{in}$

Filters : C_{out} filters of size $k \times k \times C_{in}$

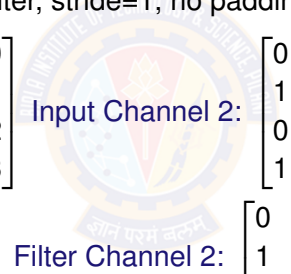
Output : $H' \times W' \times C_{out}$

Total parameters : $(k \times k \times C_{in} + 1) \times C_{out}$

MULTI-CHANNEL EXAMPLE: NUMERICAL

Setup:

- Input: $4 \times 4 \times 2$ (2 channels)
- Filter: $3 \times 3 \times 2$ (must match input depth)
- Output: $2 \times 2 \times 1$ (one filter, stride=1, no padding)



Input Channel 1: $\begin{bmatrix} 1 & 2 & 3 & 0 \\ 4 & 5 & 6 & 1 \\ 7 & 8 & 9 & 2 \\ 0 & 1 & 2 & 3 \end{bmatrix}$ Input Channel 2: $\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$

Filter Channel 1: $\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$ Filter Channel 2: $\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$

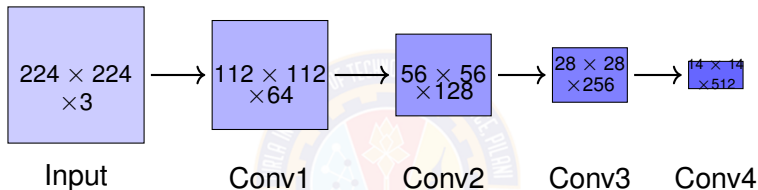
Computation at position (0,0):

$$\text{Sum from Ch1} = 1(1) + 2(0) + 3(-1) + 4(1) + 5(0) + 6(-1) + 7(1) + 8(0) + 9(-1) =$$

$$\text{Sum from Ch2} = 0(0) + 1(1) + 0(0) + 1(1) + 0(0) + 1(1) + 0(0) + 1(1) + 0(0) = 4$$

CHANNEL EVOLUTION IN DEEP CNNs

Typical pattern: Increase channels while decreasing spatial dimensions



Why this pattern?

- **Early layers:** High resolution, few channels (local patterns)
- **Deep layers:** Low resolution, many channels (semantic features)
- Maintains roughly constant **computational cost** per layer
- Total information capacity: $H \times W \times C$ approximately constant

MULTI-CHANNEL CONVOLUTION: KEY TAKEAWAY

- 1 Input channels: Filter depth = Input depth (must match)
- 2 Output channels: Number of filters = Number of output channels
- 3 Parameters: $(k \times k \times C_{in} + 1) \times C_{out}$
- 4 Computation: Each filter operates on all input channels simultaneously

Important Relationships:

- One filter $\rightarrow$ One output channel (feature map)
- More filters $\rightarrow$ More output channels $\rightarrow$ Network can learn more diverse features
- Typical progression: Channels increase with depth (3 $\rightarrow$ 64 $\rightarrow$ 128 $\rightarrow$ 256 $\rightarrow$ 512)

Memory consideration: Deep networks trade spatial resolution for channel depth.

$$\text{Memory} \propto H \times W \times C$$

POOLING OPERATIONS

What Pooling Layers Do

- **Spatial Downsampling:** Reduce height and width dimensions but preserve channel depth.
- **Translation Invariance:** Make features less sensitive to small shifts.
- **Computational Efficiency:** Reduce number of computations in subsequent layers.
- **Overfitting Reduction:** Act as a form of regularization.
- Pooling layers are **parameter-free** but still important for network performance
- Global Average Pooling (GAP) is used to replace fully connected layers.
- Two types
 - 1 Max pooling
 - 2 Average pooling

MAX POOLING

- Takes maximum value in each region
- Most common pooling operation
- Preserves strongest activations

$$\text{output} = \max(\text{inputs in window})$$

Input 4×4

2	4	7	4
6	6	9	8
3	4	8	3
7	5	3	6

Max pooling

$$\rightarrow k = 2$$
$$s = 2$$

Output

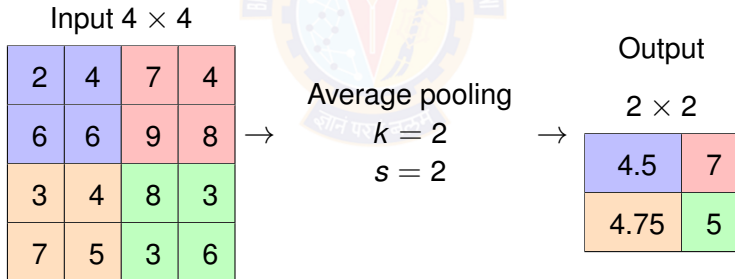
2×2

6	9
7	8

AVERAGE POOLING

- Takes average of values in region
- Smoother downsampling
- Used in final layers sometimes

$$\text{output} = \frac{1}{k^2} (\text{inputs in window})$$



1×1 CONVOLUTION: INTRODUCTION

DEFINITION

A 1×1 convolution uses a kernel of size $1 \times 1 \times C_{in}$ to perform a weighted sum across channels at each spatial location.

At First Glance:

- Seems pointless: No spatial information, just a single pixel!
- But actually very powerful: "Network in Network"

What it does:

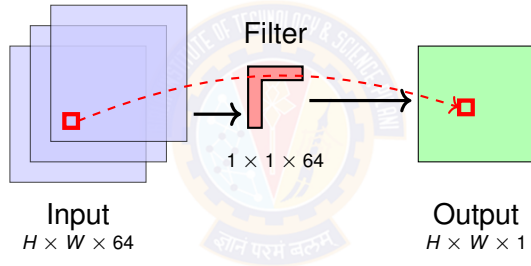
- Combines information across channels
- Acts as a learned linear combination
- Does NOT look at neighboring pixels
- Applied independently at each spatial position

1×1 CONVOLUTION: HOW IT WORKS

Input: $H \times W \times C_{in}$

Kernel: $1 \times 1 \times C_{in}$

Output: $H \times W \times 1$ (for one filter)



At each spatial position (i, j) :

$$\text{Output}(i, j) = \sum_{c=1}^{C_{in}} \text{Input}(i, j, c) \cdot w_c + b$$

1×1 CONVOLUTION: KEY APPLICATIONS

1. Dimensionality Reduction (Channel Reduction)



Reduces channels from 256 $\rightarrow$ 64 while preserving spatial dimensions.

2. Adding Non-linearity

Input $\xrightarrow{1 \times 1 \text{ Conv}}$ Linear combination $\xrightarrow{\text{ReLU}}$ Non-linear features

3. Computational Efficiency

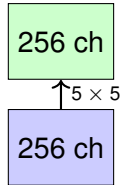
- Reduce channels before expensive 3×3 or 5×5 convolutions
- Used extensively in Inception and ResNet architectures

1×1 CONVOLUTION: BOTTLENECK ARCHITECTURE

Example: Reducing computation in deep networks

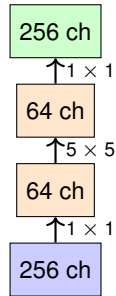
With 1×1 Conv:

Without 1×1 Conv:



Parameters:

$$= 5 \times 5 \times 256 \times 256 = 1,638,400$$



Parameters:

$$= 256 \times 64 + 5^2 \times 64^2 + 64 \times 256 = 147,456$$

11× reduction in parameters! This is the "bottleneck" design pattern.

ReLU ACTIVATION FUNCTION

DEFINITION

ReLU (Rectified Linear Unit) is a simple and powerful activation function that outputs the input directly if positive, otherwise outputs zero.

Key Properties

- **Non-linear:** Enables learning complex patterns
- **Sparse:** Many outputs are exactly zero
- **Computationally efficient:** Simple max operation
- **Non-saturating:** No upper bound on positive values

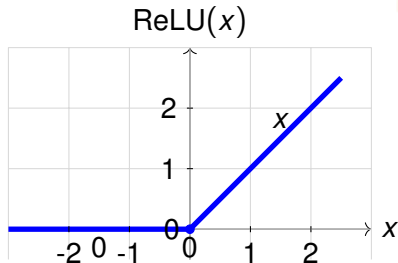
Usage in CNNs ReLU is the **default choice** for activation functions in CNNs and is typically applied after each convolutional layer:

Conv Layer Output $\rightarrow$ ReLU $\rightarrow$ Next Layer

ReLU ACTIVATION FUNCTION

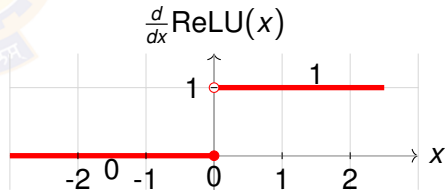
ReLU Function Graph

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$



ReLU Derivative Graph

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$$



ReLU ACTIVATION FUNCTION

Advantages of ReLU

- **Solves Vanishing Gradient Problem:** Gradient is either 0 or 1 (no saturation for positive values)
- **Computational Efficiency:** No expensive operations (exp, log, etc.)
- **Sparse Representation:** Naturally produces sparse networks
- **Biological Plausibility:** Similar to neuron firing patterns
- **Scale Invariant:** $\text{ReLU}(\alpha x) = \alpha \cdot \text{ReLU}(x)$ for $\alpha > 0$

Disadvantages of ReLU

- **Dying ReLU Problem:** Neurons can become permanently inactive (output always 0)
- **Not Zero-Centered:** Output is always non-negative
- **Non-Differentiable at Zero:** Can cause issues in some optimization scenarios

CONVOLUTION + RELU + MAX POOLING

Input Image 6×6

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Filter

-1	1	-1
-1	1	-1
-1	1	-1

Convolve, $s=1$

-1	-1	-1	-1
-1	-1	-2	-1
-1	-1	-2	1
-1	0	-4	3

Apply ReLU

0	0	0	0
0	0	0	0
0	0	0	1
0	0	0	3

Maxpool

$k=2, s=2$

0	0
0	3

FULLY CONNECTED (DENSE) LAYER

DEFINITION

A **Fully Connected (FC) Layer** or **Dense Layer** is a layer where every neuron is connected to every neuron in the previous layer, performing a linear transformation followed by an activation function.

Role in CNN Architecture

- **Feature Flattening:** Convert spatial features to vector:
 $(H \times W \times C) \rightarrow (H \cdot W \cdot C)$
- **Classification Head:** Map features to class scores
- **Dimensionality Control:** Adjust feature vector size
- **Non-linear Mapping:** Learn complex decision boundaries
- **Typical CNN Flow:**

Conv Layers $\rightarrow$ Flatten $\rightarrow$ FC Layer(s) $\rightarrow$ Output

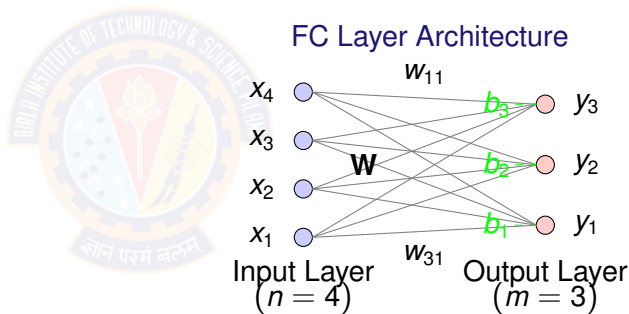
FULLY CONNECTED (DENSE) LAYER

$$\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b})$$

- $\mathbf{x} \in \mathbb{R}^n$: Input Vector
- $\mathbf{W} \in \mathbb{R}^{m \times n}$: Weight matrix
- $\mathbf{b} \in \mathbb{R}^m$: Bias vector
- $f(\cdot)$: Activation function
- $\mathbf{y} \in \mathbb{R}^m$: Output vector

Parameter Count

- **Weights:** $n \times m$ parameters
- **Biases:** m parameters
- **Total:** $n \times m + m = m(n + 1)$



FC LAYER - NUMERICAL EXAMPLE

Given: Input $\mathbf{x} = [2, -1, 3]$, weights and biases for 2 output neurons

Weight Matrix: $\mathbf{W} = \begin{bmatrix} 0.5 & -0.2 & 0.1 \\ -0.3 & 0.4 & 0.2 \end{bmatrix}$, Bias: $\mathbf{b} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$

Step 1: Linear transformation

$$z_1 = 0.5(2) + (-0.2)(-1) + 0.1(3) + 0.1 = 1.0 + 0.2 + 0.3 + 0.1 = 1.6$$

$$z_2 = (-0.3)(2) + 0.4(-1) + 0.2(3) + (-0.2) = -0.6 - 0.4 + 0.6 - 0.2 = -0.6$$

Step 2: Apply activation (ReLU)

$$y_1 = \text{ReLU}(1.6) = 1.6 \quad (1)$$

$$y_2 = \text{ReLU}(-0.6) = 0.0 \quad (2)$$

Result: $\mathbf{y} = [1.6, 0.0]$

Parameters: $3 \times 2 + 2 = 8$ total parameters

FC - ADVANTAGES AND DISADVANTAGES

Advantages:

- **Universal Approximation:** Can learn any function
- **Flexible Output Size:** Any input $\rightarrow$ any output dimension
- **Non-linear Transformations:** Complex mappings
- **Feature Integration:** Combines all spatial information

Disadvantages:

- **Parameter Heavy:** Many parameters $\rightarrow$ overfitting risk
- **Spatial Information Loss:** No spatial structure preservation
- **Fixed Input Size:** Requires consistent input dimensions
- **Computational Cost:** Matrix multiplication overhead

Modern Alternatives

- **Global Average Pooling (GAP):** Replace FC with average pooling
- **1×1 Convolutions:** Maintain spatial structure while reducing channels

SOFTMAX ACTIVATION FUNCTION

DEFINITION

Softmax is an activation function that converts a vector of real numbers into a probability distribution, commonly used in the output layer for multi-class classification.

Usage in CNNs Softmax is typically used in the **final layer** for multi-class classification:

Final Conv/FC Layer → Softmax → Class Probabilities

SOFTMAX ACTIVATION FUNCTION

For input vector $\mathbf{z} = [z_1, z_2, \dots, z_K]$:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

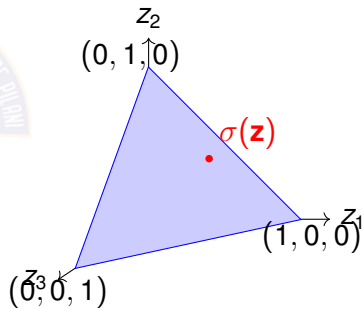
Where:

- z_i is the i -th element of input vector
- K is the number of classes
- Output: $\sigma(\mathbf{z}) = [\sigma_1, \sigma_2, \dots, \sigma_K]$

Derivative

$$\frac{\partial \sigma_i}{\partial z_j} = \begin{cases} \sigma_i(1 - \sigma_i) & \text{if } i = j \\ -\sigma_i \sigma_j & \text{if } i \neq j \end{cases}$$

Softmax Visualization (3 Classes)



2-Simplex (Valid outputs)

SOFTMAX - NUMERICAL EXAMPLE

- Given logits: $\mathbf{z} = [2.0, 1.0, 0.1]$ for 3 classes
- Step 1: Calculate exponentials

$$\begin{bmatrix} e^{z_1} \\ e^{z_2} \\ e^{z_3} \end{bmatrix} = \begin{bmatrix} e^{2.0} \\ e^{1.0} \\ e^{0.1} \end{bmatrix} = \begin{bmatrix} 7.389 \\ 2.718 \\ 1.105 \end{bmatrix}$$

- Step 2: Calculate sum

$$\sum_{j=1}^3 e^{z_j} = 7.389 + 2.718 + 1.105 = 11.212$$

- Step 3: Calculate softmax probabilities

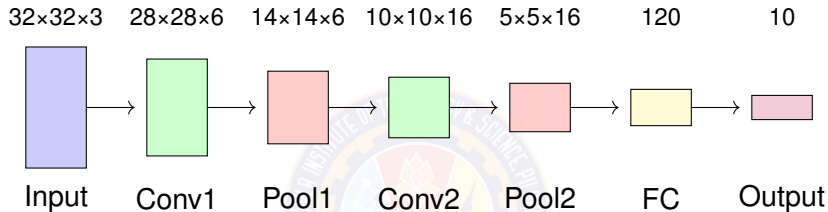
$$\begin{bmatrix} \sigma_1 \\ \sigma_2 \\ \sigma_3 \end{bmatrix} = \begin{bmatrix} \frac{7.389}{11.212} \\ \frac{2.718}{11.212} \\ \frac{1.105}{11.212} \end{bmatrix} = \begin{bmatrix} 0.659 \\ 0.242 \\ 0.099 \end{bmatrix}$$

TABLE OF CONTENTS

- 1 COMPUTER VISION TASKS
- 2 IMAGE BASICS AND IMAGE CLASSIFICATION CHALLENGES
- 3 INTRODUCTION TO CONVOLUTIONAL NETWORKS
- 4 CNN COMPONENTS
- 5 CNN ARCHITECTURE DESIGN**
- 6 POPULAR CNN ARCHITECTURES
- 7 TRANSFER LEARNING
- 8 CNN APPLICATIONS
- 9 IMPLEMENTATION CONSIDERATIONS
- 10 SUMMARY



TYPICAL CNN ARCHITECTURE



Pattern: CONV → POOL → CONV → POOL → FC → OUTPUT

- Early layers: Detect low-level features (edges, corners)
- Middle layers: Combine features (shapes, textures)
- Later layers: High-level concepts (objects, faces)
- Spatial size decreases, depth increases

NUMBER OF PARAMETERS IN CONVOLUTIONAL LAYER

For a convolutional layer with:

- Kernel size: $K \times K$
- Input channels: C_{in}
- Output channels (number of filters): C_{out}
- Bias: Yes/No, Where bias = 1 if bias is used, 0 otherwise

$$\text{Total Parameters} = (K \times K \times C_{in} + \text{bias}) \times C_{out}$$

- Parameters are independent of input spatial dimensions (height, width)
- Each filter has the same depth as input channels
- Number of parameters grows quadratically with kernel size
- Adding more filters increases parameters linearly

NUMBER OF PARAMETERS IN CONVOLUTIONAL LAYER

- Example 1: First Convolutional Layer

- ▶ Input: RGB image ($224 \times 224 \times 3$)
- ▶ Kernel size: 5×5
- ▶ Number of filters: 64
- ▶ Bias: Yes

- Calculation:

- ▶ Parameters per filter
 $= 5 \times 5 \times 3 + 1 = 76$
- ▶ Total parameters
 $= 76 \times 64 = \mathbf{4,864}$



- Example 2: Deeper Convolutional Layer

- ▶ Input: Feature map ($56 \times 56 \times 128$)
- ▶ Kernel size: 3×3
- ▶ Number of filters: 256
- ▶ Bias: Yes

- Calculation:

- ▶ Parameters per filter
 $= 3 \times 3 \times 128 + 1 = 1,153$
- ▶ Total parameters
 $= 1,153 \times 256 = \mathbf{295,168}$

NUMBER OF PARAMETERS IN POOLING LAYERS

KEY INSIGHT

Pooling layers have ZERO learnable parameters!

Why Zero Parameters?

Pooling operations are **fixed mathematical functions**: max or average

- No weights or biases to learn
- Only hyperparameters: pool size k and stride s

NUMBER OF PARAMETERS IN POOLING LAYERS

Max Pooling Example

- Input: $(56 \times 56 \times 128)$
- Pool size: 2×2
- Stride: 2
- Output: $(28 \times 28 \times 128)$

Parameters: 0

Average Pooling Example

- Input: $(56 \times 56 \times 128)$
- Pool size: 2×2
- Stride: 2
- Output: $(28 \times 28 \times 128)$

Parameters: 0



DESIGN PATTERNS FOR DEEP NETWORKS

How do we design effective deep CNN architectures?

Key Design Patterns (Common to successful architectures):

- ① Spatial reduction + Channel expansion
- ② Repetitive building blocks
- ③ Skip/residual connections
- ④ Bottleneck designs
- ⑤ Multi-scale feature extraction

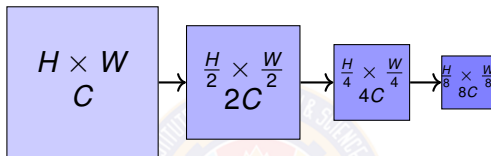
Goal: Balance between:

- Model capacity (expressiveness)
- Computational efficiency
- Memory requirements
- Training stability



PATTERN 1: SPATIAL REDUCTION + CHANNEL EXPANSION

Core Principle: As we go deeper, reduce spatial dimensions and increase channels



Rationale:

- Early: High resolution for capturing fine details
- Later: More channels for complex semantic features
- Maintains computational balance: $H \times W \times C \approx \text{constant}$

Implementation:

- Pooling layers (max/average)
- Strided convolutions (learnable downsampling)
- Typically double channels when halving spatial dimensions

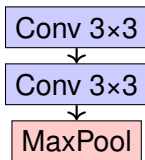
PATTERN 2: REPETITIVE BUILDING BLOCKS

Instead of arbitrary layers, use repeated modular blocks

VGG Block:

Multiple 3×3 convs + pooling

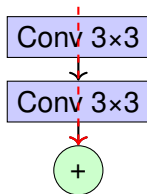
VGG Block



ResNet Block:

Convs + skip connection

ResNet Block

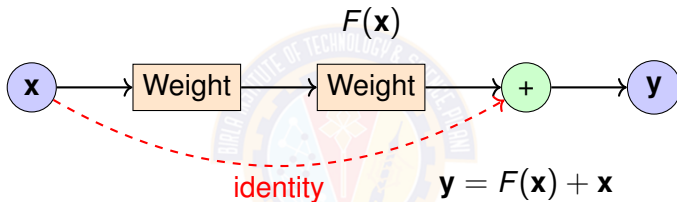


Benefits: Modularity, easier to scale, proven designs

PATTERN 3: SKIP/RESIDUAL CONNECTIONS

Problem: Deep networks suffer from vanishing gradients

Solution: Add identity shortcuts (skip connections)



Key Benefits:

- Easier gradient flow during backpropagation
- Network can learn identity mapping if needed
- Enables training of very deep networks (100+ layers)
- Used in ResNet, DenseNet, U-Net

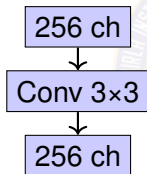
PATTERN 4: BOTTLENECK DESIGNS

Goal: Reduce computational cost without sacrificing performance

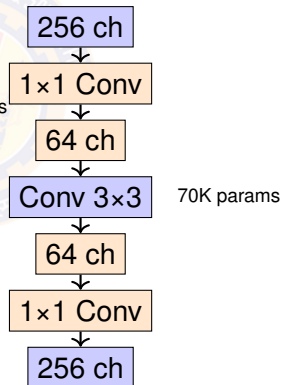
Strategy: Use 1×1 convolutions to reduce dimensions before expensive operations

8× parameter reduction! Used in ResNet-50/101/152, Inception, MobileNet

Standard:



Bottleneck:

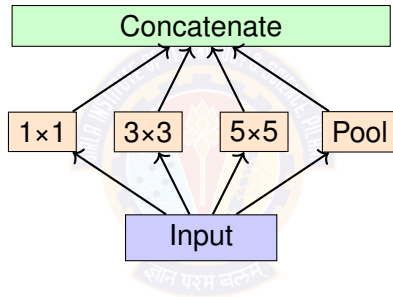


590K params

70K params

PATTERN 5: MULTI-SCALE FEATURE EXTRACTION

Idea: Process input at multiple scales simultaneously
Inception Module Example:



Benefits:

- Capture features at different scales
- More robust to object size variations
- Network decides which scale is important

HOW TO CHOOSE ARCHITECTURE DEPTH

Key Considerations:

① Task complexity

- ▶ Simple: MNIST → 5-10 layers sufficient
- ▶ Complex: ImageNet → 50-152 layers
- ▶ Very complex: Medical imaging → 100+ layers

② Dataset size

- ▶ Small dataset ($< 10K$): Shallow network (avoid overfitting)
- ▶ Medium (10K-100K): Moderate depth (20-50 layers)
- ▶ Large ($> 1M$): Very deep (50-200 layers)

③ Computational budget

- ▶ Mobile/edge: MobileNet, EfficientNet (10-20 layers)
- ▶ Desktop/server: ResNet-50, VGG-16 (16-50 layers)
- ▶ Data center: ResNet-152, EfficientNet-B7 (100+ layers)

Start shallow, add depth if underfitting and you have enough data

ARCHITECTURE DEPTH: PRACTICAL GUIDELINES

Empirical observations:

Network	Layers	Params	Use Case
LeNet-5	5	60K	MNIST, simple tasks
AlexNet	8	60M	ImageNet (2012)
VGG-16	16	138M	High accuracy, features
ResNet-50	50	25M	General purpose
ResNet-152	152	60M	State-of-art accuracy
EfficientNet-B0	18	5M	Mobile, efficient

Trends:

- Modern architectures: 50-200 layers with skip connections
- Efficient architectures: 10-30 layers with smart design
- Depth alone is not enough: Need proper training techniques

Warning: Very deep networks require:

- Batch normalization

TRADE-OFFS: DEPTH VS. WIDTH

Depth: Number of layers (vertical stacking)

Width: Number of channels per layer (horizontal expansion)

Aspect	Deeper (More Layers)	Wider (More Channels)
Feature complexity	Higher	Moderate
Receptive field	Larger	Same
Parameters	Sub-linear growth	Quadratic growth
Training difficulty	Harder (vanishing grad)	Easier
Overfitting risk	Lower (if regularized)	Higher
Inference speed	Slower (sequential)	Faster (parallel)

Example:

- **ResNet-50:** Deep (50 layers), moderate width (256-2048 channels)
- **Wide ResNet:** Fewer layers (16-28), very wide (up to 10× channels)

Current consensus: Depth + Width both matter; optimal ratio depends on task

DEPTH VS. WIDTH: PRACTICAL RECOMMENDATIONS

- When to go deeper:
 - ▶ Large, diverse datasets (ImageNet, COCO)
 - ▶ Complex tasks requiring hierarchical features
 - ▶ When you can use skip connections
 - ▶ Sufficient computational resources
- When to go wider:
 - ▶ Smaller, more homogeneous datasets
 - ▶ Need faster training/inference
 - ▶ Limited to shallow networks (no residual connections)
 - ▶ Ensemble-like behavior (different filters learn different patterns)
- Hybrid approach (EfficientNet):
 - ▶ Scale depth, width, AND resolution together
 - ▶ Use compound scaling: $\text{depth}^d \times \text{width}^w \times \text{resolution}^r = 2$
 - ▶ Achieves better accuracy-efficiency trade-off

DESIGN DECISION FRAMEWORK

① Start with baseline

- ▶ Use proven architecture (ResNet, EfficientNet)
- ▶ Transfer learning if possible

② Estimate requirements

- ▶ Dataset size → depth constraint
- ▶ Computational budget → width/depth trade-off
- ▶ Input resolution → receptive field needs

③ Iterate systematically

- ▶ Underfitting? → Add depth or width
- ▶ Overfitting? → Regularize (dropout, data aug) before reducing capacity
- ▶ Too slow? → Reduce width or use efficient blocks

④ Apply design patterns

- ▶ Always: Spatial reduction + channel expansion
- ▶ Deep networks: Skip connections mandatory
- ▶ Tight budget: Bottleneck designs

TABLE OF CONTENTS

- 1 COMPUTER VISION TASKS
- 2 IMAGE BASICS AND IMAGE CLASSIFICATION CHALLENGES
- 3 INTRODUCTION TO CONVOLUTIONAL NETWORKS
- 4 CNN COMPONENTS
- 5 CNN ARCHITECTURE DESIGN
- 6 POPULAR CNN ARCHITECTURES**
- 7 TRANSFER LEARNING
- 8 CNN APPLICATIONS
- 9 IMPLEMENTATION CONSIDERATIONS
- 10 SUMMARY



LENET-5 (1998)

ARCHITECTURE

INPUT $\rightarrow$ CONV $\rightarrow$ POOL $\rightarrow$ CONV $\rightarrow$ POOL $\rightarrow$ FC $\rightarrow$ FC $\rightarrow$ OUTPUT

- **Purpose:** Handwritten digit recognition (MNIST)
- **Key Features:**
 - ▶ First successful CNN architecture
 - ▶ Used sigmoid/tanh activations
 - ▶ Subsampling layers (average pooling)
- **Parameters:** 60K
- **Input:** 32×32 grayscale images

LeNET-5 ARCHITECTURE

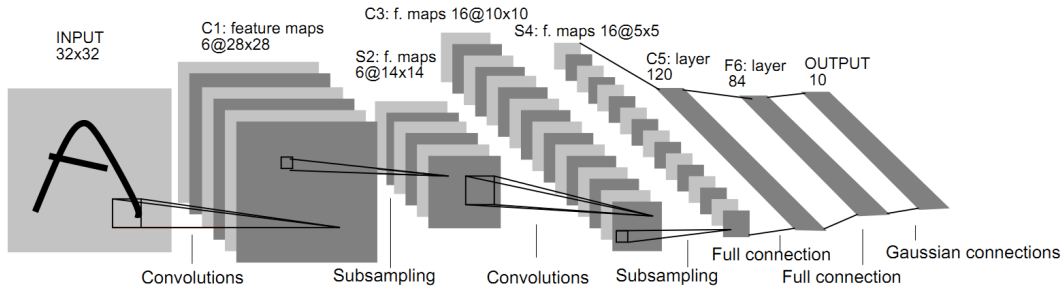


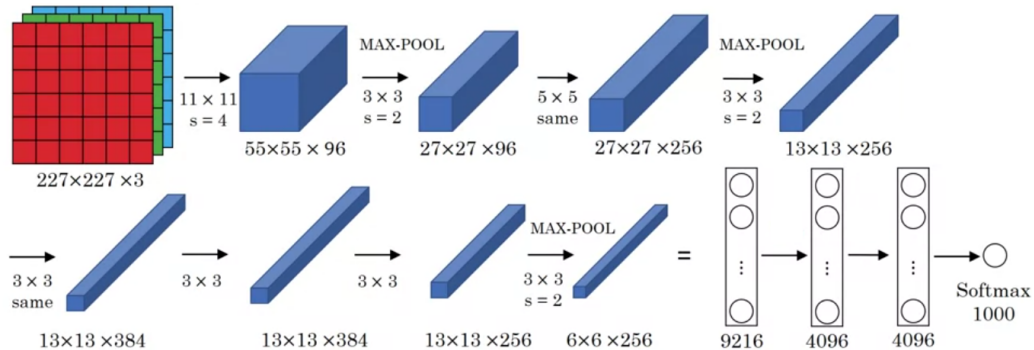
Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

ALEXNET (2012)

KEY INNOVATIONS

- Used ReLU activations (instead of sigmoid/tanh)
 - Dropout for regularization
 - Data augmentation
 - GPU acceleration
 - Local Response Normalization (LRN)
-
- **Architecture:** 8 layers (5 CONV + 3 FC)
 - **Parameters:** 60M
 - **Input:** $224 \times 224 \times 3$
 - **Achievement:** Won ImageNet 2012 (Top-5 error: 15.3)
 - **Impact:** Sparked the deep learning revolution

ALEXNET ARCHITECTURE



[Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks]

Andrew Ng

VGGNet (2014)

KEY PRINCIPLES

- Very small (3×3) convolution filters
- Deep networks (16-19 layers)
- Simple and uniform architecture
- Only 3×3 conv and 2×2 pooling throughout

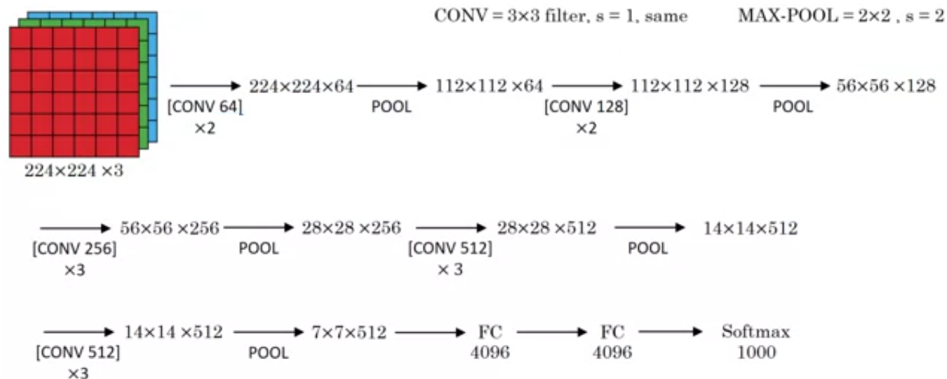
VGG-16 Configuration:

- 13 CONV layers + 3 FC layers
- All CONV layers: 3×3 filters, stride 1, pad 1
- All pooling: 2×2 max pool, stride 2

Advantages:

- Simple, homogeneous architecture
- Two 3×3 conv $\approx$ one 5×5 conv (fewer parameters)
- More non-linearities

VGGNET ARCHITECTURE



[Simonyan & Zisserman 2015. Very deep convolutional networks for large-scale image recognition]

Andrew Ng

NETWORK IN NETWORK (NiN) - 2013

CORE INNOVATION

Replace linear convolution filters with "micro neural networks" (MLPconv layers) to increase model discriminability with fewer parameters.

Key Ideas:

- **MLPconv:** Mini multi-layer perceptron instead of linear filter
- **1×1 Convolutions:** Add non-linearity and cross-channel pooling
- **Global Average Pooling:** Replace fully connected layers
- **Reduces overfitting:** Fewer parameters than traditional CNNs

Impact:

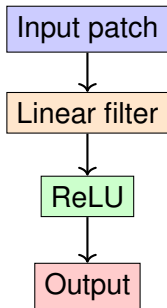
- Introduced 1×1 convolutions to deep learning
- Inspired Inception and other modern architectures
- Global average pooling became standard practice

NiN: MLP CONV LAYER

Traditional Convolution:
Linear filter + ReLU

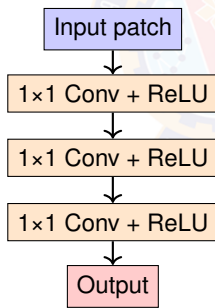
$$y = \max(0, w^T x + b)$$

Traditional Conv



MLPconv (NiN):
Multi-layer perceptron applied to local patch

MLPconv (NiN)



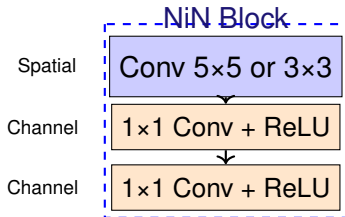
$$h_1 = \max(0, w_1^T x + b_1)$$

$$h_2 = \max(0, w_2^T h_1 + b_2)$$

$$y = \max(0, w_3^T h_2 + b_3)$$

NiN: ARCHITECTURE STRUCTURE

NiN Block = Conv layer + two 1×1 conv layers



Complete NiN Architecture:

Layer	Configuration	Output Size
Input	RGB image	$224 \times 224 \times 3$
NiN Block 1	11×11 conv, 96 filters	$54 \times 54 \times 96$
MaxPool	3×3, stride 2	$26 \times 26 \times 96$
NiN Block 2	5×5 conv, 256 filters	$26 \times 26 \times 256$
MaxPool	3×3, stride 2	$12 \times 12 \times 256$
NiN Block 3	3×3 conv, 384 filters	$12 \times 12 \times 384$
MaxPool	3×3, stride 2	$5 \times 5 \times 384$
NiN Block 4	3×3 conv, 1000 filters	$5 \times 5 \times 1000$
Global Avg Pool	Average each channel	$1 \times 1 \times 1000$
Softmax	Classification	1000 classes

NiN: GLOBAL AVERAGE POOLING

- Problem with FC layers:

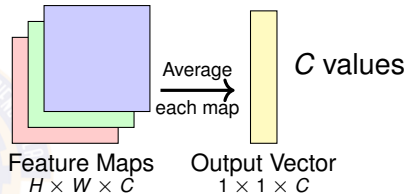
- ▶ Millions of parameters (prone to overfitting)
- ▶ Fixed input size requirement
- ▶ No spatial correspondence

- For each channel c :

$$\text{Output}_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W \text{FeatureMap}_c(i, j)$$

- **Benefits:** No parameters! More robust, no overfitting from FC layers

Global Average Pooling (GAP):



NIN: KEY TAKEAWAYS

CONTRIBUTIONS

- ① **1×1 Convolutions:** First to extensively use for adding non-linearity
- ② **MLPconv:** More expressive local models
- ③ **Global Average Pooling:** Eliminates FC layers, reduces overfitting
- ④ **Fewer Parameters:** Despite deeper, fewer params than AlexNet

Impact on Later Architectures:

- **GoogLeNet:** Adopted 1×1 convs in Inception modules
- **ResNet:** Uses 1×1 convs in bottleneck blocks
- **MobileNet:** Core building block is 1×1 convolutions
- **Modern CNNs:** GAP is now standard instead of FC layers

GOOGLENET/INCEPTION (2014) - OVERVIEW

KEY INNOVATION: INCEPTION MODULE

Use multiple filter sizes in parallel to capture features at different scales, then concatenate results.

Motivation:

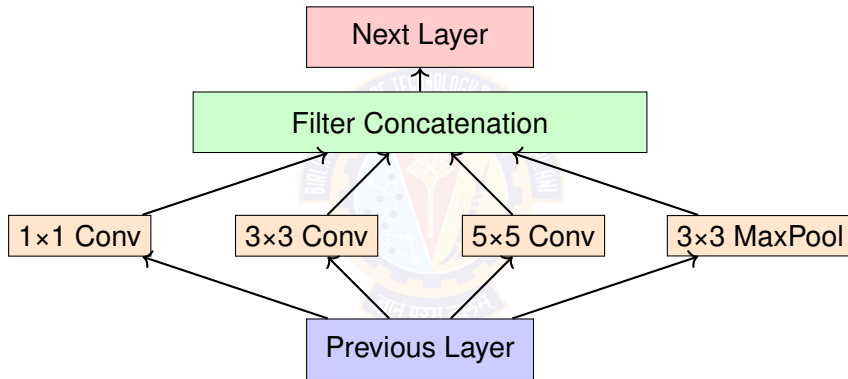
- Objects appear at different scales in images
- Hard to choose optimal filter size (1×1 ? 3×3 ? 5×5 ?)
- Solution: Use all of them!

Key Achievements:

- Winner: ImageNet ILSVRC 2014 (6.67% top-5 error)
- Depth: 22 layers (very deep for 2014)
- Parameters: 5M (12× fewer than AlexNet!)
- Architecture: Stack of inception modules

NAIVE INCEPTION MODULE

Initial Idea: Parallel paths with different receptive fields

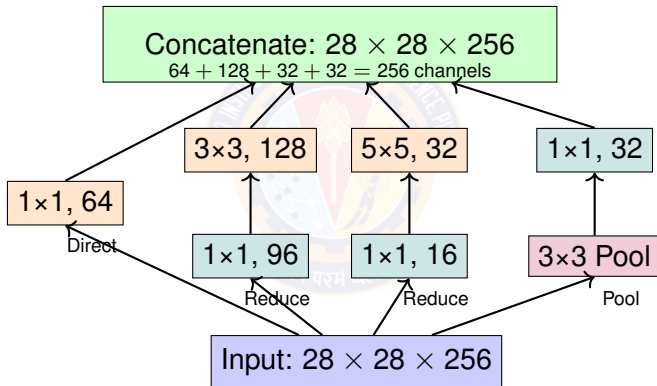


Problem: Computational cost explodes!

Example: Input 256 channels, output 256 channels each path → Millions of operations

INCEPTION MODULE WITH DIMENSIONALITY

Solution: Add 1×1 convolutions before expensive operations



Key Insight: 1×1 convs reduce channels before expensive 3×3 and 5×5 operations

INCEPTION MODULE: COMPUTATIONAL SAVINGS

- Without 1×1 reduction (naive):

- ▶ 3×3 conv: $28 \times 28 \times 256 \times 128 \times 3 \times 3 \approx 230M$ ops
- ▶ 5×5 conv: $28 \times 28 \times 256 \times 32 \times 5 \times 5 \approx 160M$ ops
- ▶ Total: $\sim 390M$ operations

- With 1×1 reduction:

- ▶ 1×1 reduce: $28 \times 28 \times 256 \times 96 \times 1 \times 1 \approx 19M$ ops
- ▶ 3×3 conv: $28 \times 28 \times 96 \times 128 \times 3 \times 3 \approx 86M$ ops
- ▶ 1×1 reduce: $28 \times 28 \times 256 \times 16 \times 1 \times 1 \approx 3M$ ops
- ▶ 5×5 conv: $28 \times 28 \times 16 \times 32 \times 5 \times 5 \approx 10M$ ops
- ▶ Total: $\sim 118M$ operations

- Result: **3.3 $\times$ reduction in computation!** This is the **bottleneck** principle that became standard in deep networks.

GOOGLENET ARCHITECTURE

Overall Structure: 22 layers deep

Layer Type	Output Size	Details
Conv 7×7, stride 2	$112 \times 112 \times 64$	
MaxPool 3×3, stride 2	$56 \times 56 \times 64$	
Conv 3×3, stride 1	$56 \times 56 \times 192$	
MaxPool 3×3, stride 2	$28 \times 28 \times 192$	
Inception (3a)	$28 \times 28 \times 256$	1st block
Inception (3b)	$28 \times 28 \times 480$	
MaxPool 3×3, stride 2	$14 \times 14 \times 480$	
Inception (4a-4e)	$14 \times 14 \times 512 - 832$	5 blocks
MaxPool 3×3, stride 2	$7 \times 7 \times 832$	
Inception (5a-5b)	$7 \times 7 \times 832 - 1024$	2 blocks
Global AvgPool	$1 \times 1 \times 1024$	NiN style
Dropout (40%)	$1 \times 1 \times 1024$	Regularization
FC + Softmax	1000 classes	Output

Total: 9 Inception modules stacked with strategic pooling for downsampling

INCEPTION MODULE: DESIGN PHILOSOPHY

Why does Inception work so well?

- ① Multi-scale feature extraction
 - ▶ Different filters capture different scales
 - ▶ Network learns which scale is important per layer
- ② Sparse connections
 - ▶ Not all inputs connected to all outputs
 - ▶ Mimics biological neural networks
- ③ Efficient computation
 - ▶ 1×1 bottlenecks reduce dimensions
 - ▶ Much fewer parameters than VGG
- ④ Increased network depth and width
 - ▶ Can go deeper without computational explosion
 - ▶ Multiple paths increase model capacity

Auxiliary Classifiers: GoogLeNet added intermediate classifiers during training to combat vanishing gradients (removed at test time)

INCEPTION VARIANTS

Evolution of Inception Architecture:

Version	Year	Key Innovation
Inception v1 (GoogLeNet)	2014	Original inception module
Inception v2	2015	Batch normalization
Inception v3	2015	Factorized convolutions ($3 \times 3 \rightarrow 3 \times 1 + 1 \times 3$)
Inception v4	2016	Combined with residual connections
Inception-ResNet	2016	Hybrid: Inception + skip connections

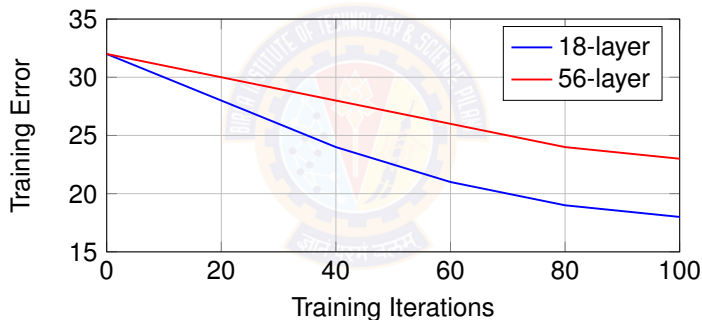
Factorization Example (v3):

- Replace expensive 5×5 with two 3×3 :
 - ▶ 5×5 conv: $25n^2$ operations
 - ▶ Two 3×3 convs: $9n^2 + 9n^2 = 18n^2$ operations
 - ▶ **Savings:** 28% fewer operations, same receptive field
- Further: Replace 3×3 with $3 \times 1 + 1 \times 3$:
 - ▶ 3×3 conv: $9n^2$ operations
 - ▶ $3 \times 1 + 1 \times 3$: $3n^2 + 3n^2 = 6n^2$ operations

RESNET (2015) - THE DEPTH PROBLEM

SURPRISING DISCOVERY

Deeper networks performed **worse** than shallow networks on training set!



Problem: Not overfitting (training error is higher!), but **optimization difficulty**

Hypothesis: Deeper networks struggle to learn even identity mappings

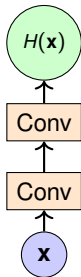
RESNET: THE RESIDUAL LEARNING IDEA

Key Insight: Instead of learning $H(\mathbf{x})$ directly, learn the residual $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$

Traditional Block:

Learn target mapping directly:

$$H(\mathbf{x}) = \text{layers}(\mathbf{x})$$

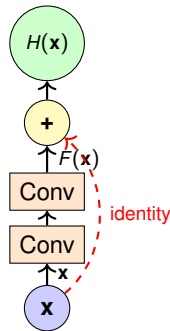


Why easier? If identity is optimal, network just needs to learn $F(\mathbf{x}) = 0$ (push weights to zero), which is easier than learning identity explicitly!

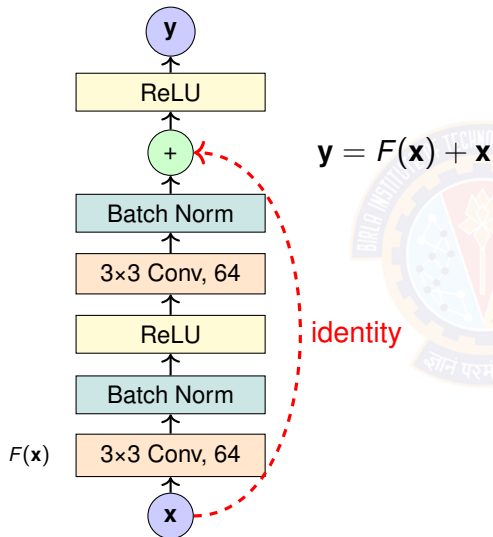
Residual Block:

Learn the difference (residual):

$$H(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}$$

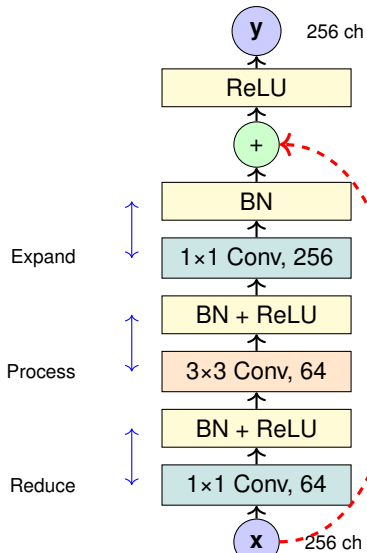


BASIC RESIDUAL BLOCK



- Building Block for ResNet-18 and ResNet-34:
- Key Components:
 - ▶ Two 3x3 convolutions
 - ▶ Batch normalization after each conv
 - ▶ ReLU activation (one in middle, one at end)
 - ▶ Identity skip connection (red dashed)

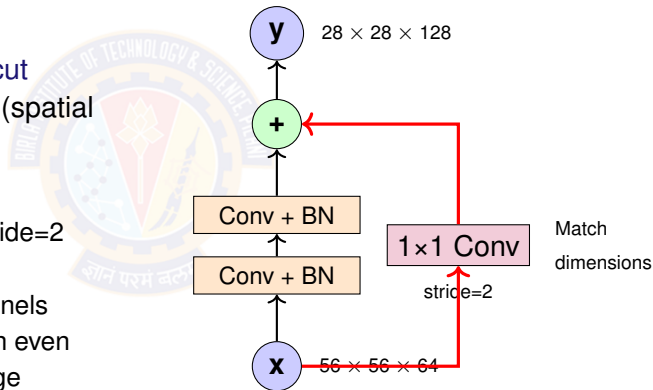
BOTTLENECK RESIDUAL BLOCK



- Used in ResNet-50, ResNet-101, ResNet-152 for efficiency:
- **Bottleneck Strategy:**
 1×1 reduce $\rightarrow 3 \times 3$ process $\rightarrow 1 \times 1$ expand
- **Parameter savings:**
 $256 \times 64 + 64 \times 64 \times 9 + 64 \times 256 = 70K$ vs. $256 \times 256 \times 9 = 590K$
(8.4 $\times$ reduction!)

RESNET: HANDLING DIMENSION MISMATCH

- **Problem:** What if input and output dimensions don't match?
- **Solution: Projection Shortcut**
When dimensions change (spatial or channel):
- **Projection:**
 - ▶ 1×1 convolution with stride=2 (downsample spatially)
 - ▶ Adjusts number of channels
 - ▶ Enables skip connection even when dimensions change



RESNET ARCHITECTURE FAMILY

Layer	ResNet-18	ResNet-34	ResNet-50	ResNet-101
Conv1	7×7, 64, stride 2			
Conv2_x	[3×3, 64] × 2/3 ×2	×3	[1×1,64; 3×3,64; 1×1,256] × 3 ×3	×3
Conv3_x	[3×3, 128] × 2/4 ×2	×4	[1×1,128; 3×3,128; 1×1,512] × 4/23 ×4	×23
Conv4_x	[3×3, 256] × 2/6 ×2	×6	[1×1,256; 3×3,256; 1×1,1024] × 6 ×6	×6
Conv5_x	[3×3, 512] × 2/3 ×2	×3	[1×1,512; 3×3,512; 1×1,2048] × 3 ×3	×3
Total Layers	18	34	50	101
Parameters	11.7M	21.8M	25.6M	44.5M
Top-5 Error	10.2%	9.6%	7.7%	7.1%

WHY DO RESIDUAL CONNECTIONS WORK?

Multiple explanations:

① Gradient Flow

- ▶ Skip connections create "gradient highways"
- ▶ Backprop: $\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot (1 + \frac{\partial F}{\partial \mathbf{x}})$
- ▶ Gradient always has direct path (the "1" term)

② Ensemble Behavior

- ▶ Network behaves like ensemble of shallow networks
- ▶ Each path through skip connections = different network
- ▶ 2^n implicit paths in n -block network

③ Feature Reuse

- ▶ Earlier features directly accessible to later layers
- ▶ No information bottleneck
- ▶ Layers can refine features instead of recreating them

RESNET: KEY TAKEAWAYS

Revolutionary Impact ResNet enabled training of networks with 100+ layers, winning ImageNet 2015 with 3.6% error (superhuman performance on ImageNet!)

Core Contributions:

- ① **Residual Learning:** Learn $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$ instead of $H(\mathbf{x})$
- ② **Skip Connections:** Identity shortcuts for gradient flow
- ③ **Very Deep Networks:** 50-152 layers trainable (even 1000+ demonstrated)
- ④ **Bottleneck Design:** Efficient deep networks using 1×1 convolutions

Impact on Field:

- Skip connections now standard in almost all architectures
- DenseNet, U-Net, EfficientNet all use residual-style connections
- Transfer learning: ResNet features widely used as backbones
- Enabled modern very deep networks (Transformers also use residuals!)

TABLE OF CONTENTS

- 1 COMPUTER VISION TASKS
- 2 IMAGE BASICS AND IMAGE CLASSIFICATION CHALLENGES
- 3 INTRODUCTION TO CONVOLUTIONAL NETWORKS
- 4 CNN COMPONENTS
- 5 CNN ARCHITECTURE DESIGN
- 6 POPULAR CNN ARCHITECTURES
- 7 TRANSFER LEARNING**
- 8 CNN APPLICATIONS
- 9 IMPLEMENTATION CONSIDERATIONS
- 10 SUMMARY



TRANSFER LEARNING CONCEPT

DEFINITION

Using a pre-trained model on one task as the starting point for a related task

Why Transfer Learning?

- **Limited Data:** Small datasets benefit from pre-trained features
- **Computational Efficiency:** Avoid training from scratch
- **Better Performance:** Often achieves better results
- **Feature Reusability:** Low-level features are transferable

Common Scenario

- **Source Task:** ImageNet classification (1.2M images, 1000 classes)
- **Target Task:** Your specific classification problem
- **Assumption:** Similar low-level visual features

IMAGENET DATA



Figure 3: 96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images. The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU

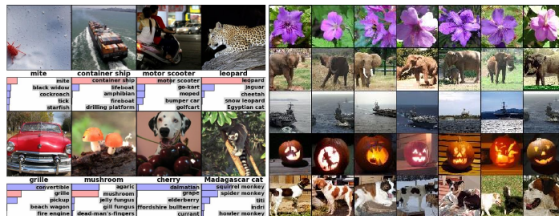
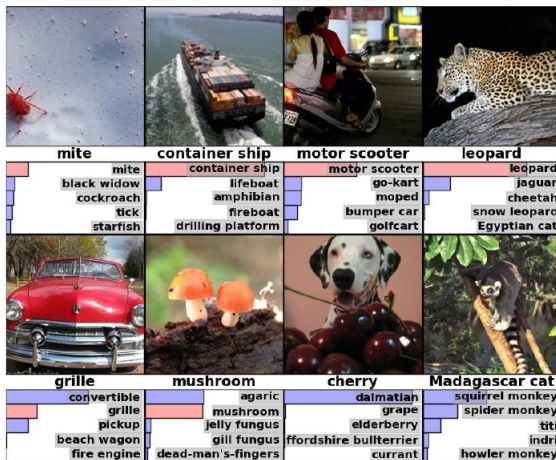


Figure 4: **(Left)** Eight ILSVRC-2010 test images and the five labels considered most probable by our model. The correct label is written under each image, and the probability assigned to the correct label is also shown with a red bar (if it happens to be in the top 5). **(Right)** Five ILSVRC-2010 test images in the first column. The remaining columns show the six training images that produce feature vectors in the last hidden layer with the smallest Euclidean distance from the feature vector for the test image.

IMAGENET DATA



The ImageNet set that was used has ~1.2 million images and 1000 classes

Accuracy is measured as top-5 performance: Correct prediction if the true label matches one of the top 5 predictions of the model

TRANSFER LEARNING STRATEGIES

Strategy 1: Feature Extraction

- Freeze pre-trained layers
- Train only classifier (final layer)
- Use when: Small dataset, similar to pre-trained data

Strategy 2: Fine-tuning

- Initialize with pre-trained weights
- Train entire network with low learning rate
- Use when: Medium dataset, somewhat similar data

Strategy 3: Selective Fine-tuning

- Freeze early layers, fine-tune later layers
- Balance between feature extraction and fine-tuning
- Use when: Medium dataset, different but related data

TRANSFER LEARNING GUIDELINES

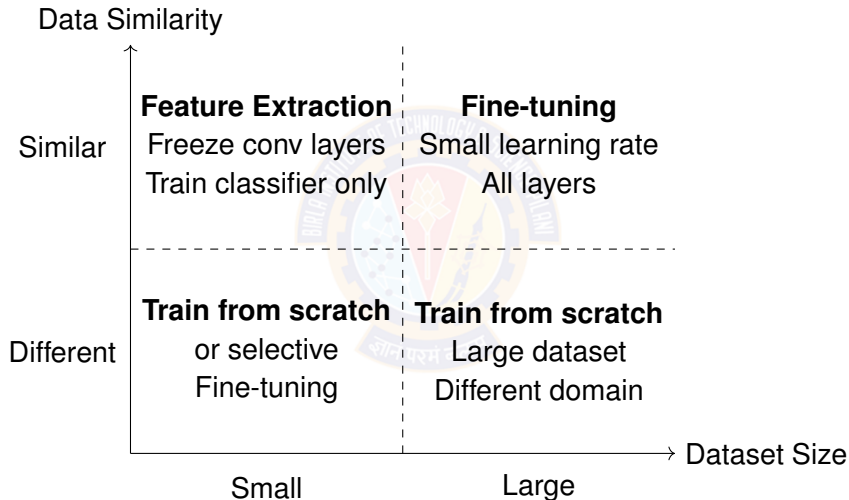


TABLE OF CONTENTS

- 1 COMPUTER VISION TASKS
- 2 IMAGE BASICS AND IMAGE CLASSIFICATION CHALLENGES
- 3 INTRODUCTION TO CONVOLUTIONAL NETWORKS
- 4 CNN COMPONENTS
- 5 CNN ARCHITECTURE DESIGN
- 6 POPULAR CNN ARCHITECTURES
- 7 TRANSFER LEARNING
- 8 CNN APPLICATIONS**
- 9 IMPLEMENTATION CONSIDERATIONS
- 10 SUMMARY



IMAGE CLASSIFICATION

TASK

Assign a single label to an entire image

- **Examples:** CIFAR-10, ImageNet, medical image diagnosis
- **Architecture:** Standard CNN $\rightarrow$ Global Average Pooling/FC $\rightarrow$ Softmax
- **Loss Function:** Cross-entropy loss
- **Metrics:** Accuracy, Top-k accuracy

KEY CONSIDERATIONS

- Data augmentation crucial for generalization
- Class imbalance handling
- Proper train/validation/test splits

OBJECT DETECTION

TASK

Locate and classify multiple objects in an image

- **Output:** Bounding boxes + class labels
- **Approaches:**
 - **Two-stage:** R-CNN, Fast R-CNN, Faster R-CNN
 - **One-stage:** YOLO, SSD
- **Challenges:** Multiple scales, varying object sizes, real-time requirements

SEMANTIC SEGMENTATION

TASK

Classify every pixel in the image

- **Output:** Dense pixel-wise predictions
- **Architectures:**
 - ▶ **FCN:** Fully Convolutional Networks
 - ▶ **U-Net:** Encoder-decoder with skip connections
 - ▶ **DeepLab:** Atrous convolutions, multi-scale features
- **Applications:** Medical imaging, autonomous driving, satellite imagery

OTHER CNN APPLICATIONS

Computer Vision Tasks

- **Face Recognition:** Identity verification, face detection
- **Style Transfer:** Artistic style application
- **Image Generation:** GANs, VAEs
- **Super-Resolution:** Enhance image quality

Beyond Computer Vision

- **Natural Language Processing:** Text classification, sentiment analysis
- **Audio Processing:** Speech recognition, music analysis
- **Time Series:** 1D convolutions for sequential data
- **Graph Neural Networks:** Graph convolutions

TABLE OF CONTENTS

- 1 COMPUTER VISION TASKS
- 2 IMAGE BASICS AND IMAGE CLASSIFICATION CHALLENGES
- 3 INTRODUCTION TO CONVOLUTIONAL NETWORKS
- 4 CNN COMPONENTS
- 5 CNN ARCHITECTURE DESIGN
- 6 POPULAR CNN ARCHITECTURES
- 7 TRANSFER LEARNING
- 8 CNN APPLICATIONS
- 9 IMPLEMENTATION CONSIDERATIONS**
- 10 SUMMARY



PRACTICAL IMPLEMENTATION TIPS

- Data Preprocessing
 - ▶ Normalize pixel values (0-1 or standardize)
 - ▶ Data augmentation: rotation, flipping, cropping, color jittering
 - ▶ Proper train/validation/test splits
- Architecture Design
 - ▶ Start simple, add complexity gradually
 - ▶ Use batch normalization for faster training
 - ▶ Apply dropout for regularization
 - ▶ Monitor validation performance
- Training Strategy
 - ▶ Use appropriate learning rate scheduling
 - ▶ Early stopping to prevent overfitting
 - ▶ Gradient clipping for stability
 - ▶ Use pre-trained models when possible

COMMON PITFALLS AND SOLUTIONS

- Overfitting
 - ▶ **Symptoms:** High training accuracy, poor validation accuracy
 - ▶ **Solutions:** More data, data augmentation, dropout, L2 regularization
- Underfitting
 - ▶ **Symptoms:** Poor performance on both training and validation
 - ▶ **Solutions:** Larger model, more training time, lower regularization
- Vanishing/Exploding Gradients
 - ▶ **Solutions:** Proper initialization, batch normalization, residual connections
- Computational Efficiency
 - ▶ Use appropriate batch sizes
 - ▶ Mixed precision training
 - ▶ Model compression techniques

TABLE OF CONTENTS

- 1 COMPUTER VISION TASKS
- 2 IMAGE BASICS AND IMAGE CLASSIFICATION CHALLENGES
- 3 INTRODUCTION TO CONVOLUTIONAL NETWORKS
- 4 CNN COMPONENTS
- 5 CNN ARCHITECTURE DESIGN
- 6 POPULAR CNN ARCHITECTURES
- 7 TRANSFER LEARNING
- 8 CNN APPLICATIONS
- 9 IMPLEMENTATION CONSIDERATIONS
- 10 SUMMARY



KEY TAKEAWAYS

Core Concepts

- CNNs exploit spatial structure through local connectivity and parameter sharing
- Convolution, pooling, and non-linear activations are building blocks
- Hierarchical feature learning: edges $\rightarrow$ shapes $\rightarrow$ objects

Architecture Evolution

- LeNet $\rightarrow$ AlexNet $\rightarrow$ VGG $\rightarrow$ GoogLeNet $\rightarrow$ ResNet $\rightarrow$ Modern architectures
- Key innovations: ReLU, dropout, batch normalization, residual connections
- Trend: Deeper networks with better optimization techniques

PRACTICAL GUIDELINES

- Start with transfer learning when possible
- Data augmentation is crucial for generalization
- Monitor both training and validation metrics
- Use appropriate regularization techniques
- Implement a simple CNN from scratch
- Experiment with different architectures on CIFAR-10
- Try transfer learning with pre-trained models
- Explore data augmentation techniques

REFERENCES

- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into deep learning*. Cambridge University Press. Chapters 6 and 7.
- Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in NIPS*.
- Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*.
- Szegedy, C., et al. (2015). Going deeper with convolutions. *Proceedings of the IEEE CVPR*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE CVPR*.

Thank You!